

UUID vs. Integer ID: Decision Guide

Decision Tree: Should I Use a UUID?

Question 1: Do you have a distributed system (microservices)?

YES → Use UUID

- Allows offline ID generation without coordination
- Prevents ID collisions between databases
- No need for centralized ID generation service
- Perfect for eventual consistency systems

Example Scenario:

None

Service A (Users) generates: 550e8400-e29b-41d4-a716-446655440000

Service B (Orders) generates: 7c9e6679-7425-40de-944b-e07fc1f90ae7

- ✓ No collision possible
- ✓ Can be generated independently
- ✓ Can merge databases later

NO → Continue to Question 2

Question 2: Do you want to hide the total number of records?

YES → Use UUID

Sequential IDs reveal business metrics:

- `user/105` reveals you have ~105 users
- `order/50123` shows ~50,000 orders
- Competitors can track growth by creating accounts

UUIDs provide privacy:

- `user/550e8400-e29b-41d4-a716-446655440000` reveals nothing
- Can't estimate database size
- Can't track growth rate

NO → Auto-increment integers are faster

- Smaller storage (4-8 bytes vs 16 bytes)
 - Better for indexing (sequential)
 - Easier to debug and reference
 - Faster JOIN operations
-

UUID Versions: Which Should I Use?

UUID v4 (Random)

Best for: Most general use cases

 Pros:

- Cryptographically random
- Globally unique without coordination
- No information leakage

 Cons:

- Not sortable by creation time
- Can cause database fragmentation
- Slightly slower inserts

Use When:

- You need distributed generation
- Creation time doesn't matter
- Security is priority

Example: `f47ac10b-58cc-4372-a567-0e02b2c3d479`

UUID v7 (Time-Sortable)

Best for: Database performance + distributed systems

 Pros:

- Includes timestamp component
- Sortable by creation time
- Better database performance (less fragmentation)

- Still globally unique

✗ Cons:

- Reveals creation time (mild info leakage)
- Newer standard (less library support)

Use When:

- You need time-ordered data
- Database performance matters
- You have high-volume inserts

Format Breakdown:

```
None
01234567-89ab-7def-0123-456789abcdef
|       |       |  └ Random
|       |       └   Version (7)
|       └   Random
└ Unix timestamp (milliseconds)
```

UUID v1 (Time + MAC Address)

⚠ Generally Avoid

✗ Cons:

- Exposes MAC address (privacy concern)
- Reveals exact creation time
- Not suitable for public-facing IDs

Only Use When:

- Internal systems where privacy doesn't matter
 - Need guaranteed uniqueness (MAC ensures it)
-

Performance Comparison

Storage Size

Type	Size	Example
Integer (32-bit)	4 bytes	2147483647
BigInt (64-bit)	8 bytes	9223372036854775807
UUID	16 bytes	550e8400-e29b-41d4-a716-446655440000
UUID (string)	36 bytes	"550e8400-e29b-41d4-a716-446655440000"

Index Performance

Sequential Integer:

-  Fastest inserts (append to end)
-  Minimal page splits
-  Best cache locality

UUID v4 (Random):

-  Random inserts cause page splits
-  Slower on large datasets (>10M rows)
-  Use with clustered indexes carefully

UUID v7 (Time-sortable):

-  Nearly as fast as sequential integers
-  Minimal page splits
-  Good for distributed systems

Database Implementation

PostgreSQL

```
SQL
-- Native UUID support
CREATE TABLE users (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
name VARCHAR(255)
);
```

MySQL

```
SQL
-- Binary storage (most efficient)
CREATE TABLE users (
  id BINARY(16) PRIMARY KEY,
  name VARCHAR(255)
);
```

MongoDB

```
JavaScript
// ObjectId is similar to UUID v1
db.users.insertOne({
  _id: new ObjectId(), // Auto-generated
  name: "John Doe"
});
```

When NOT to Use UUIDs

✗ Use Auto-Increment Instead When:

1. Single database system

- No microservices
- No data replication needs
- Centralized architecture

2. Small datasets (<1M rows)

- Performance difference negligible
- Debugging easier with integers
- User-facing IDs (invoice numbers)

3. URL requirements

- Short URLs matter (bit.ly-style)
- User needs to type ID
- Marketing campaigns

4. Financial systems

- Sequential order required
- Audit trails need clear ordering
- Regulatory compliance

Hybrid Approach

Best Practice: Use Both

```
SQL
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_number SERIAL UNIQUE, -- Human-readable
  email VARCHAR(255),
  created_at TIMESTAMP
);
```

- **UUID:** Internal references, API responses
- **Serial Number:** User-facing ID, support tickets

Example:

- API: `GET /users/550e8400-e29b-41d4-a716-446655440000`
- Support: "What's your user number?" → "User #10523"

Common Mistakes

✗ Storing UUIDs as Strings

```
SQL
-- BAD: 36 bytes per ID
id VARCHAR(36)
```

SQL

```
-- GOOD: 16 bytes per ID
id BINARY(16) -- MySQL
id UUID      -- PostgreSQL
```

✗ Using UUID v1 in Public APIs

Exposes MAC address and timestamp → privacy issue

✗ Not Using UUID v7 for Time-Series Data

Random UUIDs cause performance issues in time-series databases

Quick Decision Matrix

Scenario	Recommendation
Microservices architecture	UUID v7
Single monolithic app	Auto-increment Integer
Need chronological sorting	UUID v7 or Auto-increment
Public-facing IDs	Auto-increment or Short ID
Offline-first apps	UUID v4 or v7
High security requirements	UUID v4
Financial/Legal records	Auto-increment (audit trail)

Generate UUIDs in Code

JavaScript:

```
JavaScript
// Using crypto (browser/Node.js 19+)
crypto.randomUUID();

// Using library
import { v4 as uuidv4, v7 as uuidv7 } from 'uuid';
const id = uuidv7(); // Recommended
```

Python:

```
Python
import uuid

# v4 (random)
id = uuid.uuid4()

# v7 (time-sortable) - Python 3.13+
id = uuid.uuid7()
```

PHP:

```
PHP
// v4
$uuid = Ramsey\Uuid\Uuid::uuid4();

// v7
$uuid = Ramsey\Uuid\Uuid::uuid7();
```

Need Expert Help?

Scaling your database architecture?

Devxhub specializes in high-performance cloud infrastructure with proper ID strategy, sharding, and replication.

 Contact us: info@devxhub.com

 Visit: www.devxhub.com

 Call: +880 1326 506464

Our Services:

- Database architecture design
- Microservices consulting
- Cloud infrastructure setup (AWS, GCP, Azure)
- Performance optimization